

RL Learning • 强化学习

Part 2 — RL Diagnosis Casebook / RL 诊断案例集

Five diagnosis problems, full back-and-forth · including wrong first calls, corrected misdiagnoses, and the path to the root

How to read this casebook. Each problem doesn't just give the "right answer" — it reconstructs the real diagnostic process: **first call (often wrong or incomplete) → challenged → corrected → traced to the root**. Diagnostic skill isn't reciting conclusions; it's seeing where you went wrong when you go wrong. So the places where "the first call was off and got corrected" are exactly what's worth studying. Structure of each: scenario → symptom → first call and detour → correction → locating criterion → fix → boomerang (new problems the fix may introduce).

Contents

- 0 Diagnosis Master Frame: four axes + on-the-spot script
- Q1 Data-Center Cooling — objective wrong (reward misspecification)
- Q2 Robot-Arm Insertion — sparse reward (exploration can't reach signal)
- Q3 Maze Navigation — representation overfitting (memorizing local visuals)
- Q4 Self-Driving BC — closed-loop mismatch (train open-loop, deploy closed-loop)
- Q5 Long-Game — credit assignment (delayed reward)
- App. Full Math of δ vs A (tower rule / Bellman expansion)

0 Diagnosis Master Frame

A learning system = objective (reward/loss) $\times$ capacity (model class) $\times$ data/environment $\times$ optimization. When it goes wrong, it's one of these (or a coupling). The core diagnostic skill is **locating the layer fast** — the four fixes are completely different and easily misdiagnosed.

On-the-spot script (facing "the model behaves strangely," don't jump to fixes; ask out loud in order):

1. Did it **learn wrong** or **learn nothing**? (skewed vs empty)
2. If wrong, did it **perfectly optimize what I wrote**? (→ reward wrong, not model wrong)
3. Did training error drop? (No → capacity/optimization; yes-but-weird → objective/data)
4. Is this hack's reward actually higher on the training distribution? (Yes → spec gaming, fix the spec)
5. Is it exploiting a real signal or an environment/evaluator bug?

Core insight: the optimizer is always right — it gives you exactly what you asked. A model "going wrong" is almost never the model's fault; it's a gap left in objective/environment/data. **First reflex: audit the spec, not add capacity.**

Q1 Data-Center Cooling — Objective Wrong

Scenario

An RL agent controls data-center cooling. action = set each unit's cooling power, reward = $-$ (total energy), goal is to save power. Hard constraint: any rack temperature above $T_{\max}$ gives a big negative reward $-M$.

Symptom

Training curve is great, 18% better than the human baseline. Stress testing reveals: the agent presses cooling to near-zero most of the time, and **only slams it on at the last instant before touching the ceiling, oscillating right against $T_{\max}$** . Energy is low, but the units cycle on/off rapidly, shortening hardware life.

First call and detour

The reflex is to say "the model learned wrong, add a penalty term." Two things get challenged here:

- **"The model learned wrong" is the wrong axis.** The agent precisely and honestly maximized $-$ energy; hugging the ceiling **really is the lowest energy** — it optimized correctly, didn't skew. Saying "learned wrong" sends you to fix the model; **the truth is the objective is missing something (no hardware life / safety margin), and you should fix the reward**. This distinction is directional; get it backward and every fix is wrong. (Aside: the oscillation isn't a "saddle point" — it's a near-optimal policy the reward endorses, i.e. a gap between reward and intent, not a saddle.)
- **"Add a temperature-rate smoothing term" penalizes the wrong quantity.** What engineers worry about is the compressor's rapid on/off — that's **action (the power**

command) jittering, not temperature. Penalize action smoothness $\|a_t - a_{t-1}\|$. (On healthy equipment where power and temperature are tightly coupled, penalizing the temperature rate can transmit indirectly — but penalizing the action is more direct and depends less on the "healthy coupling" assumption. That's a robustness trade-off, not right vs wrong.)

Correction: symptom vs root cause

Even if you add smoothness correctly (penalize action), it's **still not enough** — the most easily missed layer:

"Oscillation" is only the symptom; the root is "running against $T_{\max}$." After adding action-smoothness, the agent's new strategy is almost inevitably **to run smoothly and steadily right against $T_{\max}$** : no more oscillation (dodging the smoothness penalty), still lowest energy (still hugging the ceiling), but the units run permanently at the edge with **zero safety margin** — any disturbance overshoots. **You cured the oscillation, not the hugging, and bred a more insidious version.**

Locating and criterion

Layer: objective wrong (reward misspecification). Criterion: the agent didn't "fail to learn" (it performs great), nor is it a capacity/optimization issue (error drops well); it learned a **high-reward but bad** policy that **really did maximize the written reward** — the fingerprint of "the objective is missing something."

Fix (three layers, none optional)

1. **action smoothness** — $\lambda_1 \|a_t - a_{t-1}\|$: cures the on/off cycling (symptom).
2. **soft barrier / soft upper bound with a buffer**: cures "hugging the ceiling" (root). **The core is replacing the cliff with a slope** — don't use a cliff-shaped "penalize only above $T_{\max}$ "; make the penalty rise increasingly and smoothly as temperature approaches $T_{\max}$, structurally removing the "hugging-the-edge is optimal" incentive.
3. **Ask again about each new term's gap**: too large a $\lambda \rightarrow$ over-conservative agent, cooling always on, energy goal dead (from one extreme to the other). The balance of λ is itself a new attack surface.

Boomerang / closing

The cliff-shaped reward ($-M$ hard threshold) is the root of all evil — it creates "as long as I don't cross, the closer the better," so the agent inevitably hugs the line. The real cure isn't patching on top, it's **reshaping the reward (cliff $\rightarrow$ slope)**. Smoothness is symptomatic; the soft barrier is the cure. **The diagnostic dividing line: can you predict on paper "what new hack will a new term breed"** — the "hugging" gap is pure logic, derivable without deploying.

Q2 Robot-Arm Insertion — Sparse Reward

Scenario

A multi-stage robot-arm task: (a) reach the part → (b) grasp → (c) insert into the socket. reward is +1 **only on successful insertion**, 0 everywhere else. Trained with standard PPO.

Symptom

After long training, the agent reliably reaches the part and then **stops, motionless**, fine-tuning its pose, never attempting a grasp. The reward curve **stays at 0**, never rising.

First call and key clarification

Intuition: "every stage needs feedback; only the end has reward; the search space is too big" — direction right. But there's a **fatal misclassification to avoid**:

It's not "the training data is insufficient." This is the fundamental difference between RL and supervised learning:

- Supervised: data is externally given and static; "insufficient data" → **collect more data**.
- RL: the agent **generates its own data** — it doesn't lack interaction; it runs countless trajectories daily. What it lacks is **data with signal**: every trajectory has reward 0, and running more still gives all zeros.

So the precise diagnosis is: **the agent generates massive data, but the data contains no effective learning signal (reward stays zero, exploration can't reach the single +1)**.

Diagnosing "insufficient data" leads to "collect more data" (wrong path, RL isn't data-starved); diagnosing "data exists but has no signal" leads to "help exploration reach the sparse signal" (right path).

One more wording fix: "the data doesn't exist" is also wrong — **data exists in bulk; what doesn't exist is the effective signal**. "Data doesn't exist" leads to collecting data (wrong); "data exists but has no signal" leads to helping exploration (right).

Locating and criterion

Layer: **exploration-signal**. Criterion: **reward curve stays at zero + agent frozen = the fingerprint of "never got off the ground"**, not of "learned wrong." The objective is fine (rewarding successful insertion is right), the reward structure isn't skewed; the signal is too sparse and far, exploration can't reach the first positive feedback, gradient is zero for a long time. Not a capacity problem (not that the model can't hold the solution — it never got a learnable signal).

Q1 vs Q2: skewed vs empty (this distinction locates it)

- **Q1:** data is **meaningful but the meaning is wrong** (agent learned a skewed policy, reward nonzero and captured by bad behavior) → objective layer, fix reward.
- **Q2:** data is **meaningless (all zero, because sparse)**, agent learned nothing → exploration layer, help exploration / add signal.

The difference = "has signal but wrong" vs "no signal at all" = skewed vs empty. See "learned wrong" → check reward encoding; see "zero and frozen" → check sparsity/exploration. **Note:** in Q2 the specific cause of "meaningless" is sparsity (the zero-symptom pins it down); don't dilute this concrete diagnosis with a general principle like "RL has lots of data but it may be useless."

Fix (one class, not just one method)

Make sparse dense / assist exploration: **reward shaping** (add intermediate signal), **intrinsic / curiosity** (reward reaching novel states), **demonstration / imitation warm-start, curriculum** (train an easier version first). Don't just say "add feedback per stage" — that's only one kind of shaping, narrowed too early.

Boomerang

Add reward shaping and **how to weight the per-stage rewards becomes a new attack surface:** weighted wrong, the agent may only farm the easy-to-score middle stage (repeatedly reaching the part for a "proximity reward" but never grasping), turning the shaping signal itself into a new hack. **The shaping introduced to solve sparsity (Q2) can create a new misspec (Q1).** The proper fix: potential-based shaping (provably doesn't change the optimal policy).

Q3 Maze Navigation — Representation Overfitting

Scenario

Agent navigates a maze. state = agent's **first-person visual image** (pixels) at its current position, action = up/down/left/right. reward = +1 at goal, -0.01 per step (encourage speed). PPO + a CNN encoding the image.

Symptom

The agent does great on the **training maze**, reliably reaching the goal. Switch to a **new maze** (different layout, same visual style) and it's **completely lost**, performance drops to near random. Training reward climbs well, no anomaly.

First, nail the setup (concepts that must be clear before diagnosis)

- **reward** is a fixed environment rule (+1 goal, -0.01 per step), **not learned, not computed on the fly**. Unlike RLHF (which trains a reward model because "human preference" has no ready formula); here the environment gives it directly, no reward model needed.
- **"Can't walk through walls" is transition** $P(s' | s, a)$ (the environment forbids passing through, state unchanged), **not reward**. You may add a negative reward for hitting a wall, but that's a design choice; "hitting a wall means staying put" is itself a transition rule.
- **V(s) is multi-step value** (this task takes many steps to the goal), estimated by a critic network $V_{\phi}(s)$ regressing the actual cumulative return. **But how V is estimated is unrelated to this diagnosis** — the symptom is "train good, new maze bad," not "V estimated poorly." Don't get sidetracked by V.

Key setup clarification: first-person local, not global (repeatedly tripped here)

The state is the local image the agent sees in front of it; the agent cannot see the global maze. This is easily misremembered as "global top-down map + coordinates." Misremember it and the causal chain goes wrong:

- ❌ Wrong version: "has global info → memorizes global maze features → new maze fails" (there's no global info, doesn't hold).
- ✅ Right version: "only first-person local vision → the agent didn't learn **navigation**, but **rote-memorized 'visual cue → action'** (see this red wall → turn right, this texture → go straight) → new maze, visual cues change, the memorized mapping fails → lost."

What overfits is the training maze's visual surface, not the global structure.

Where the CNN sits: shared by policy and value (another easy error)

It's not a "global CNN," nor "CNN only used in value." Standard practice (when state is an image):

```
local image → [CNN backbone] → feature vector h
                                   ↳ [policy head] → action prob  $\pi(a|s)$ 
                                   ↳ [value head]  →  $V(s)$ 
```

The CNN backbone is **shared by policy and value**, with two heads on top. **This shared structure is exactly where the disease lives: if the shared CNN learns a representation that only memorizes the training maze's surface visuals, the policy head and value head fail together — both stand on that non-generalizing feature h. The root is in the shared CNN representation, not in either head.**

Not feature engineering — representation learning failure

A natural follow-up: "shouldn't be this fragile — is the feature engineering bad?" — direction (chasing finer mechanism) right, wording wrong:

- **Deep RL doesn't do manual feature engineering** — the CNN's whole point is learning features from pixels automatically. So the problem isn't "feature engineering done badly," it's **the automatically-learned representation isn't transferable**.
- It's not "some weight too large," it's **the whole representation learned in the wrong direction**: it learned "memorize what this maze looks like," not "understand spatial structure for navigation."

Locating, criterion, root cause

Layer: representation/generalization. Criterion: **train good + new maze bad = what's learned doesn't generalize = representation overfitting**. Not the objective layer (reward is fine), not exploration (it clearly learned something, does great on the training maze).

Root cause: training distribution too narrow (only the training maze) + no generalization pressure. Rote-memorizing the training maze's visuals already maxes the reward, so the CNN has no incentive to learn a transferable representation — memorizing is the least-effort solution. Not feature engineering, not weight magnitude — a single training distribution makes overfitting the least-effort solution.

Two root causes for "train good, test bad" in RL

The same symptom in RL can be two different diseases (easy to lock onto "overfitting" and miss the second):

1. **Representation overfitting** (main here): CNN memorizes training-maze visuals, **perception** doesn't generalize (can't tell what to do on a new image). This is the classic overfitting supervised learning also has.
2. **Policy memorizing an action sequence** (RL-specific secondary variant): the agent may learn a **fixed action sequence** for the training maze's specific layout ("left three, right two"), **policy** doesn't generalize (never learned "decide action based on what I see," learned a rote routine).

Different fixes: the first fixes the representation (data augmentation, domain randomization, stronger encoder); the second fixes policy learning (procedurally generate many mazes to prevent rote routines).

Fix

Give it generalization pressure. Core: **domain randomization** — procedurally generate many different mazes. See enough mazes and rote-memorizing is no longer least-effort; the CNN is forced to learn "general navigation" rather than "memorize one maze." Plus data augmentation (randomize visual style, lighting) to make the representation robust to surface changes.

Boundary with Q4 (plant a line)

Q3 is **static generalization**: switch environment (really change the map) and it breaks — the distribution gap is **external**. Q4 next is **dynamic compounding**: same environment, self-amplifying over time. Don't lump both under "generalization."

Q4 Self-Driving BC — Closed-Loop Mismatch

Scenario

Simplified self-driving. state = front-camera image, action = steering angle. Use **imitation-learning warm-start**: collect lots of **human-driver** data (s, a_{human}), supervised-learn the policy to imitate human actions (behavior cloning), then fine-tune with RL.

Symptom

In the BC phase, validation-set action prediction is **very accurate** (predicted angles nearly match humans, low loss). But once the policy **actually drives** (closed loop), the car **gradually drifts from the lane center, further and further, finally off the road** — even on a straight road seen countless times in training.

First call: analogy to A/B-test that fails in production (half right)

Intuition: "like picking a model via A/B and it flops in production — validation set differs from reality" — big frame right (offline good, online bad = distribution mismatch), but it **misses the RL layer of "self-creating the difference"**:

Two kinds of distribution mismatch, different mechanisms:

- **Ordinary drift / A/B (passive)**: the distribution changed because **the external world changed** (users, season); the model is a victim, didn't create the drift.
- **Q4 closed-loop mismatch (active, RL-specific)**: in the BC validation set the states are all **human-driven** (humans always stay at lane center, so training states are all "car centered"). But when the policy drives itself, **its actions have small errors → the car drifts slightly → it faces a state never seen in training → predicts worse → drifts more → state more unfamiliar → error snowballs → off the road.**

This mismatch isn't the external world changing — the policy's own actions drove it into states the training set never covered. An agent-created distribution drift (covariate shift / compounding error), not passively encountered. Supervised learning doesn't have this — samples are iid, one bad prediction doesn't affect the next input; in closed loop this step's error **changes the next step's input**.

Correction 1: is the "validation set bad"? — no, a principled blind spot

A sharp rebuttal: "if it never validated states like a drifted car, the validation set is badly made" — conclusion is backward:

Those drifted states simply don't exist at data-collection time — not "left out of validation," but "uncreatable." BC data comes from human drivers who **always keep the car centered**, and won't deliberately drift then correct just to sample it. So human data inherently lacks "the car has drifted" states; a validation set carved from the same human data is also all "car centered" → same distribution as training → accurate prediction is **genuinely accurate**, it **didn't lie**.

It's not that the validation set slacked — any offline validation has a principled blind spot in closed-loop tasks: the states the policy creates can't be produced at collection time. This isn't bad engineering, it's the essential gulf between closed and open loop. To cover those states you don't change the validation set — you change the **data-collection method** (Dagger: let the policy drive to produce drifted states, then ask a human "how to recover here").

Correction 2: "this sensitive? not the whole story" — right, three factors stacked

Doubt: "a slight drift is a generalization issue — really this sensitive?" — the intuition is right; a single "not covered" can't explain the snowball. The real collapse stacks three:

1. **(a) out-of-distribution + limited generalization:** drift a bit, the network can still roughly extrapolate, slightly worse prediction. **Alone not fatal.**
2. **(b) closed-loop compounding — the snowball's engine:** the key isn't "drift a bit, predict a bit worse," it's that the slightly-off action **drifts the car more** → **next state more unfamiliar** → **predict worse** → **drift more**. Error **feeds the next step, positive-feedback amplified**, a small error recursed into exponential divergence by the closed loop. **Supervised learning doesn't have this.**
3. **(c) no correction samples:** human data has no "I drifted, steer back to recover" corrective action (humans don't drift, so no correction samples). Once drifted the policy **doesn't know how to recover**, only drifts on.

Three stacked cause the snowball: (b) is the engine, (a)(c) the fuel. None alone drives off the road.

Correction 3: "train and validate are both supervised, this isn't RL" — right, and that's the point

The behavior-cloning phase is pure supervised learning (input state, fit human action, no reward, no environment interaction, no policy gradient). Resolving "how is the policy deployed": **the supervised-trained network IS the policy; putting it on the car to run closed-loop is deployment**. Same network, two run modes:

- **Training (open loop)**: s comes from the human dataset (fixed), each prediction independent, iid, validation accurate.
- **Deployment (closed loop)**: s comes from the car's actual state, which is determined by the previous action; π 's output becomes the next input, error accumulates.

The root is the "train open-loop, deploy closed-loop" mode mismatch — deeper than "distribution mismatch": mismatch of distribution is the result, mismatch of run mode (open vs closed loop) is the cause.

Locating and final closing: this isn't RL's failure, it's the failure of NOT using RL

Layer: distribution drift / closed-loop mismatch (method-problem nature mismatch). Criterion: **offline metrics good, online bad, and it collapses on a straight road seen countless times, accumulating over time** (not breaking on a switched environment — that's Q3).

Final conclusion (say this in an interview): this isn't RL's failure — it's the failure of **not using RL**. BC is open-loop supervised learning, while driving is a closed-loop sequential-decision task (the policy's actions change the states it later encounters). The method's assumptions (iid, open loop) mismatch the task's nature (non-iid, closed loop), yielding covariate shift and compounding error. **This problem proves RL's irreplaceable value in sequential decisions: RL is inherently closed-loop; its training process itself experiences "my action changes my next state."**

Fix: make training closed-loop too

- **Dagger**: let the policy drive itself (produce drifted states) → ask a human "how to recover in this drifted state" → add "policy-created states + human corrective actions" to training. No RL, but turns **data collection** from open to closed loop.
- **RL fine-tuning**: the policy drives itself and learns from environment reward, naturally experiencing the closed loop, learning to recover.

Q3 vs Q4 boundary (nailed)

Q3 (static generalization): the world changed (really a new map), external distribution gap → fix representation generalization (domain randomization).

Q4 (dynamic compounding): same straight road, the policy drives itself off, internal distribution gap compounding over time → fix "make training closed-loop" (DAgger / RL).

Q3 is "the world changed" (external), Q4 is "drove itself off" (internal, closed-loop compounding).

Q5 Long-Game — Credit Assignment (delayed reward)

Scenario

One long strategy game (200 steps). reward only at game end: win +1, lose -1, all 199 middle steps are 0. PPO + GAE.

Symptom

The agent learns slowly (win rate 50%→60%), but **extremely slow and unstable**, the win-rate curve oscillating violently. Diagnosis shows the agent often **assigns credit/blame to the wrong step**: a game won, the +1 credit gets **spread or misassigned** to some irrelevant middle actions (even opening random moves), while the truly key decision (the step-150 brilliancy) isn't reinforced.

Q2 vs Q5: two different "sparse" diseases

Q2 and Q5 are both "sparse reward," but **different diseases**:

- **Q2:** sparse to the point that **exploration can't reach the +1**, the signal is simply unobtainable → can't learn (reward stays zero).
- **Q5:** the signal is **obtainable** (reaches the end, win rate rising), but **the delay is too long + horizon too long, so credit can't be accurately assigned to the responsible step** → learns but slow and misassigned.

One is "can't reach the signal," the other is "reached it but can't tell whose credit it is."

First call: the disease is in the value network (direction right, mechanism to fill)

Intuition: "converging means value is useful, else it couldn't learn; the problem must be value making some states' scores weird, not learned right" — **direction right** (the disease is V estimated poorly), but the mechanism needs spelling out.

How does credit travel from the step-200 +1 back to the step-150 brilliancy? Entirely via the value network as a relay. Middle steps have $r_t = 0$, so

$$\delta_t = r_t + vV(s_{t+1}) - V(s_t) = vV(s_{t+1}) - V(s_t)$$

Credit is entirely determined by the **value difference of adjacent states**. For the brilliancy to get credit, we need $V(\text{after brilliancy}) > V(\text{before})$, i.e. the value network must see that "this step improved the position."

Correction: not "the step's action signal is gone" — the signal travels entirely via V , and V is estimated poorly

An easy misstatement: " δ_t averaged over action and future state, so the step's reinforce/weaken signal is gone" — **backward**.

- In $\gamma V(s_{t+1})$, s_{t+1} is the next state the step's action a_t **actually led to**, so **the step's action signal isn't lost — it's hidden in "which s_{t+1} you reached"**: good action $\rightarrow$ good s_{t+1} (high V) $\rightarrow \delta_t > 0 \rightarrow$ reinforced; bad action $\rightarrow$ bad $s_{t+1} \rightarrow \delta_t < 0 \rightarrow$ weakened.

The real disease: middle steps have no real reward anchor, the signal is entirely entrusted to V_ϕ ; over a long horizon V_ϕ is estimated poorly, δ_t distorts $\rightarrow$ misassignment. That +1 must travel 200 steps, decayed by $(\gamma\lambda)^{50}$ to reach step 150; decay + noise leaves the signal weak and blurry at middle steps; V_ϕ then **statistically spreads** "the win" across all states along the way (correlation mistaken for causation), and the key step isn't identified.

Why "spread" + why oscillation: GAE's bias-variance dilemma

The gradient is $\sum_t \delta_t \nabla \log \pi(a_t|s_t)$ (small λ). δ_t is each action's weight. V_ϕ estimated wrong $\rightarrow$ weights wrong $\rightarrow$ irrelevant actions get positive weight (reinforced), the brilliancy near-zero weight (not reinforced) = misassignment. And middle-step $\delta_t = \gamma V(s_{t+1}) - V(s_t)$ are all small noisy differences (real reward is 0, entirely from V -estimate jitter), **extremely low SNR** $\rightarrow$ gradient direction dominated by noise $\rightarrow$ oscillation.

GAE's λ is trapped in a dilemma:

- λ small (toward TD, relies on V_ϕ): V_ϕ inaccurate $\rightarrow \delta_t$ biased $\rightarrow$ credit misassigned.
- λ large (toward MC, relies on actual return): doesn't touch V_ϕ but the 200-step return's **variance explodes** $\rightarrow$ training oscillates.

Long horizon + sparse reward forces GAE into a dilemma: either bias (misassignment) or variance (oscillation). The symptoms "slow + oscillating + misassigned" are exactly this dilemma.

Locating and criterion

Layer: credit assignment / delayed reward. Criterion: **the reward isn't so sparse it can't learn (Q2's stuck-at-zero); it learns but slow and misassigned** — win rate slowly climbing (some credit did travel right), but the curve oscillates violently and credit spreads to

irrelevant steps. This points to "signal obtainable, but the relay carrying it (V_ϕ) is estimated poorly over a long horizon, misassigning credit."

Fix (two ideas; a is the cure)

a (reward shaping / densify middle reward) — the cure: the root is middle-step $r_t = 0$, making δ_t rely purely on V -estimate differences with no real anchor. a injects real reward into middle steps (e.g. "capture a piece +small, hold key ground +small"), making $r_t \neq 0$, so $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ has a real anchor $\rightarrow$ credit no longer relies purely on inaccurate V . **"reward=0 is the root, so make reward $\neq 0$ " — hits directly.**

b (improve propagation / shorten horizon) — not the cure: however you improve propagation (large λ , better V_ϕ , hierarchical), **middle-step reward is still 0**, the signal is still sparse and must travel far. Large λ bypasses V_ϕ but brings variance explosion (still oscillates). b wrestles the root; a removes it.

(Note: the right reason to reject b is "doesn't cure the root," not "don't know the horizon" — "improve propagation" is pure algorithm-level, needing no horizon knowledge; "shorten horizon" is what needs domain knowledge to split the task. Don't conflate the two.)

Boomerang: a introduces a Q1-style misspec

Add reward shaping and **how to weight it becomes a new attack surface (back to Q1):** weighted wrong, the agent may **only farm the easy middle scores** (repeatedly capturing pieces for small points) without pursuing the final win, turning the shaping signal itself into a new hack. **a cures Q5 (credit assignment) but may introduce Q1 (reward misspec).**

The proper fix: **potential-based reward shaping** — using shaping of the form $F(s, s') = \gamma \Phi(s') - \Phi(s)$, which **provably doesn't change the optimal policy**, only speeds learning. This is the only way to "densify reward without introducing a new hack." Add this sentence when answering a in an interview and it's watertight.

Appendix Full Math of δ vs A (traced to the bottom)

Q5's core is understanding exactly how **TD residual** δ_t and **advantage** A_t differ. This section traces "they're both expectations, where's the difference?" down to the algebra.

First, distinguish three quantities: reward / Q / V

- r_t (**reward**): the instant, single-step score, given on the spot by the environment. Middle steps $r_t = 0$, end ± 1 .
- $Q(s, a)$: starting from s , **forced to take action a this step**, then following the policy, the expected total return **all the way to the end**. **Q is not reward** — reward is only Q's first term: $Q = r_t + \gamma(\text{rest})$.
- $V(s)$: starting from s , **averaging over actions per the policy**, the expected total return all the way to the end.

The only difference between Q and V: **whether the first action is fixed**. Relation:

$$V(s) = \sum_a \pi(a | s) Q(s, a) = E_{a \sim \pi}[Q(s, a)].$$

Why Q must be "to the end," not single-step reward: the brilliancy's reward is also 0, but its value lies in "leading to the end's +1" — this "value pointing to the future" is only captured by the to-the-end Q. If Q were just reward, middle-step advantages would all be 0 and the agent could never learn middle decisions.

Definitions of advantage and δ

$$A_t = Q(s_t, a_t) - V(s_t) \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Expand Q: $A_t = E[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots | s_t, a_t] - V(s_t)$.

The precise location of the difference (from step $t + 1$ on):

- A_t uses the **real** $r_{t+1} + \gamma r_{t+2} + \dots$ to the end (doesn't touch $V_\phi \rightarrow$ unbiased, but needs a full trajectory $\rightarrow$ high variance).
- δ_t uses **the single number** $V(s_{t+1})$ to summarize all of "from $t + 1$ to the end" (only one real step r_t , the rest entrusted to $V \rightarrow$ biased, but doesn't need to run to the end $\rightarrow$ low variance).

δ_t is a **single-step bootstrap estimate** of A_t ($\lambda = 0$ GAE), not A_t itself.

"They're both expectations, where's the difference?" — expand V, find equal expectations

Expand $V(s_{t+1})$ in δ_t by definition (it's itself an expectation):

$V(s_{t+1}) = E[r_{t+1} + \gamma r_{t+2} + \dots | s_{t+1}]$. Apply the **tower rule (law of total expectation)**

$E[E[X | Y]] = E[X]$ over s_{t+1} :

$$E[V(s_{t+1}) | s_t] = E[r_{t+1} + \gamma r_{t+2} + \dots | s_t]$$

Tower rule intuition (group-average then average = total average): group heights by gender — boys' mean 175 (60%), girls' mean 165 (40%),
 $E[E[\text{height} \mid \text{gender}]] = 0.6 \times 175 + 0.4 \times 165 = 171 = \text{overall mean}$. Inner is "a quantity varying by group," outer averages it by group probability, equaling the total average.
 Here "gender" = s_{t+1} , "height" = total return from $t + 1$, inner conditional average = $V(s_{t+1})$, outer average by transition probability.

Pure proof: $E[\delta_t] = 0$ (under true V)

First prove the Bellman equation (from definition + tower rule):

$$V(s_t) = E[\sum_{k \geq 0} \gamma^k r_{t+k} \mid s_t] = E[r_t + \underbrace{\sum_{k \geq 1} \gamma^k r_{t+k}}_{\text{tower rule over } s_{t+1}} \mid s_t] = E[r_t + \gamma V(s_{t+1}) \mid s_t]$$

Main proof (take conditional expectation of δ_t , split by linearity):

$$E[\delta_t \mid s_t] = E[r_t + \gamma V(s_{t+1}) \mid s_t] - \underbrace{E[V(s_t) \mid s_t]}_{\text{by Bellman} = V(s_t)} = \underbrace{V(s_t)}_{= V(s_t) \text{ (const given } s_t)} - V(s_t) = 0$$

Similarly $E[A_t] = 0$: $E_{a_t \sim \pi}[A_t \mid s_t] = E_{a_t}[Q(s_t, a_t) \mid s_t] - V(s_t) = V(s_t) - V(s_t) = 0$ (using $V = E_{a \sim \pi}[Q]$).

Conclusion: under true V , $E[\delta_t] = E[A_t] = 0$; at the expectation level there's no difference (both zero). Intuition: advantage is "how much better than average," so averaging over actions is of course zero; the residual is "how much better than expected," so when the expectation is accurate the average is also zero.

The difference lies entirely in: the real world uses the estimate V_ϕ , not the true V .

- A_t (actual return, $\lambda = 1$): future uses real samples, **doesn't touch** $V_\phi \rightarrow$ unbiased, high variance.
- δ_t (V_ϕ , $\lambda = 0$): future replaced by $V_\phi(s_{t+1}) \rightarrow$ **biased** by V_ϕ 's error, low variance.

When V_ϕ is perfect the two are identical; only when imperfect do they diverge. **GAE's λ tunes "trust actual return vs trust V_ϕ "** — that's the bias-variance trade-off, not in the expectation itself but in "which way you estimate the same expectation, and whether V_ϕ is accurate."

GAE's telescoping sum (why $\lambda = 1$ returns to actual return)

$$A_t^{\text{GAE}(\lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l+1}$$

At $\lambda = 1$, $\sum_l \gamma^l \delta_{t+l+1} = \sum_l \gamma^l (r_{t+l+1} + \gamma V(s_{t+l+1}) - V(s_{t+l}))$; adjacent $\gamma^{l+1} V(s_{t+l+1})$ and $-\gamma^{l+1} V(s_{t+l+1})$ **cancel pairwise** (telescoping), leaving only $\sum_l \gamma^l r_{t+l+1} - V(s_t) = \text{actual return}$

– $V(s_t)$. At $\lambda = 0$ it's just δ_t . GAE interpolates between the two ends.

*End of Part 2 — the full back-and-forth of five diagnosis problems. The core of diagnostic skill isn't reciting conclusions,
it's seeing where you went wrong when your first call is off: skewed vs empty, external vs internal, symptomatic vs root, and every fix's boomerang.*